

PERFORMANCE AUDIT • BOURSENEGAR

BourseNegar Production Stack

The platform is slow because of one query pattern against one untuned table — not because of Flask. Here is what I measured against your live database, and the stack that actually fixes it.

14 August 2026

Flask 3 • PostgreSQL 17 • Jinja + vanilla JS

StockPlatform_Claude

Your framework is not the bottleneck. A market page spends roughly **5,900 ms** inside PostgreSQL and under **2 ms** inside Flask.

Rewriting in FastAPI, Go, or Node would move the number you are unhappy about by about 0.03%. The work is in the data layer and it is very fixable — I



work is in the data layer, and it is very important to have tested the fix on your own data before writing this.

What I measured

I connected to your stock database and benchmarked `_gainer()` — the query behind `/stocks` , `/etfs` , and every other market-wide page — then rebuilt the same result as a precomputed snapshot and timed that.

ROWS SCANNED	TABLE SIZE	LIVE QUERY	PRECOMPUTED
6,119,261	1,263 MB	5,884 ms	8.9 ms
SNAPSHOT SIZE			
184 kB			

TODAY – COMPUTED PER REQUEST5,884 ms

Warm run. The first run after a restart took 13,908 ms.

PRECOMPUTED – READ FROM A SNAPSHOT8.9 ms

Identical output: 782 tickers, same periods, same numbers. Bars drawn to scale.

Same result, **661x faster** – and the snapshot costs 6.8 s to rebuild, once per data update, not once per click.

Here is why the live query is expensive. This is the real plan from your database:

```
GroupAggregate (actual rows=782 loops=1)
  Buffers: shared hit=441 read=135779, temp read=3879 written=389
  -> WindowAgg (actual rows=852573 loops=1)
    -> Gather Merge   Workers Launched: 2
      -> Sort (actual rows=284191 loops=3)
        Sort Key: ticker, j_date DESC
        Sort Method: external merge  Disk: 10400kB
        -> Parallel Seq Scan on stockpricehistory
          Filter: ((adj_final > '0'::numeric)
            AND ((j_date)::text <= '1405-05-18'::text)
            AND ((j_date)::text >= '1403-05-18'::text))
          Rows Removed by Filter: 1755563
Execution Time: 5852.726 ms
```

EXPLAIN (ANALYZE, BUFFERS) – STOCKPRICEHISTORY, 2-YEAR WINDOW

Three things are happening on every single request: Postgres reads **135,779 blocks — about 1.06 GB, the entire table**; it throws away 1,755,563 of the rows it just read; and it sorts the survivors on disk because it has no memory to sort in.

Diagnosis

ROOT CAUSE

Analytics are recomputed from raw prices on every request

`_gainer()` window-functions a two-year slice of the whole price table to produce 782 rows of returns. No index can rescue this — the query touches every ticker, so a sequential scan is the *correct* plan. The problem is not the plan. It is that the work happens at request time at all, when the underlying prices only change once a day.

`/performance` is worse: its all-time query scans the full table with no date cutoff, then runs a `LATERAL` first-row lookup per ticker. Your own comment in `db.py` notes it is too heavy to warm.

CONFIGURATION

PostgreSQL is running on stock defaults

Every setting below is the out-of-the-box value, sized for a machine far smaller than yours and a database far smaller than this one.

SETTING

NOW

SHOULD

shared_buffers

128 4 GB
MB

work_mem

4 64 MB
MB

effective_cache_size

4 12 GB
GB

random_page_cost

4.0 1.1

max_parallel_workers_per_gather

2 4



SILENT

The planner is working from statistics that are wrong by 2,700×

`pg_stat_user_tables` reports **2,282 live rows** for `stockpricehistory`. The real count is **6,119,262**. Autovacuum has never successfully analyzed this table, so every plan Postgres chooses — join order, scan type, memory allocation — is decided against numbers that are meaningless. Running `ANALYZE` is a one-line fix and it should be the first thing you do.

SCHEMA

The column types work against the queries

- `j_date` is `varchar(10)`, so every date range filter is a **string comparison across 6.1 million rows** — and a real `date` column already sits in the same table, unused by the analytics.
- Every price column is `numeric`, cast with `::float` at query time, six million times per scan.
- `ix_sph_ticker_jdate` and `ix_sph_jdate` exist but this access pattern cannot use them.

MULTI-USER

Five things break the moment a second worker starts

This is the part that matters most for launch, because none of it fails on your laptop with one user — it fails the first time two people click at once.

- `_CACHE = {}` is a plain in-process dict — unbounded, no TTL, no eviction. Four Unicorn workers means four independent copies in RAM and four simultaneous warm-up scans fighting over the same disk.
- `warm_cache()` is already throttled in your code because it saturates Postgres. Multiply that by every worker you add.
- Job control lives in files — `update_stop.flag` , `update_job.meta.json` . A stop click that lands on worker 2 cannot stop a job running inside worker 1.
- The updater is a bare `threading.Thread` . It dies with the process, with no retry, no queue, and no history.
- `app.run(debug=True)` is the Werkzeug development server. Single process, and the interactive debugger is remote code execution if it is ever exposed

is ever exposed.

The stack

Most of this is *keep*. You do not have a technology problem, so the recommendation is not a rewrite — it is four additions and one deletion.

LAYER	TODAY	SHIP WITH	WHY
Web framework	Flask 3	Flask 3 — keep	Contributes ~2 ms of you 5,900 ms. Replacing it changes nothing.
UI	Jinja + vanilla JS	Keep	RTL, charts, Jalali datepicker al working with no dependencie A React rewrite buys you nothing here.
Analytics	computed	Materialized	The whole

Analytics	computed per request	materialized views	the whole ballgame: 5,884 ms → 8.9 ms.
Cache	in-process dict	Redis	One cache shared by every worker with TTL and eviction, surviving restarts.
App server	Werkzeug debug	Gunicorn (gthread)	Real concurrency. Threads, not processes — the workload waits on Postgres, it does not burn CPU.
Edge	none	Nginx	TLS, brotli/gzip, static files, rate limiting.

Jobs	thread + flag files	Celery + Beat	The TSE fetch survives restarts, retries on failure, and has one scheduler instead of one per worker.
Database	PostgreSQL 17, defaults	PostgreSQL 17, tuned	Right database. Wrong settings. See above.
Migrations	none	Alembic	Once real users have rows, schema changes need a safe path.
Deploy	manual, Windows	Docker Compose, Linux	One file describes the whole system rebuilds are repeatable.

Observability`print()`**Sentry +
JSON logs**You cannot f
what users h
silently.

Migration order

This sequence is deliberate — each step is safe to ship on its own, and the speed arrives before the infrastructure work does.

01 **Run ANALYZE, then tune postgresql.conf**

No code changes at all. Correct statistics plus real memory settings will take a meaningful bite out of the 5,884 ms before you have written a line.

~30 MINUTES • ZERO RISK • IMMEDIATE

02 **Materialized views for the market-wide analytics**

One view per heavy table — returns, performance, strategy, filter, and score scans. Refresh them `CONCURRENTLY` at the end of each data update so reads never block. Then delete `warm_cache()` entirely: warming stops being per-process RAM and becomes a database artifact every worker shares.

~1 DAY • THE SINGLE BIGGEST WIN • 5,884 MS → 8.9 MS

03 **Schema pass**

Filter on the existing `date` column and keep `j_date` for display only. Convert `adj_*` to double precision. Add `(ticker, date DESC) INCLUDE (adj_final)` for the single-ticker detail and chart lookups.

~HALF A DAY • SPEEDS UP THE REFRESH ITSELF

04 **Redis replaces the in-process dict**

Swap `_CACHE` for Redis with a version key bumped on each data update. This is the step that makes running more than one worker safe.

~HALF A DAY • UNBLOCKS MULTI-USER

05 **Gunicorn behind Nginx, in Docker Compose, on Linux**

App, Postgres, Redis, worker, scheduler, and Nginx as one compose file. `debug=True` goes away here.

~1 DAY • THIS IS THE ACTUAL LAUNCH STEP

06 **Celery and Beat for the TSE fetch**

Move job state out of `update_stop.flag` and `update_job.meta.json` and into Postgres. Beat runs the nightly fetch; the last task in the chain refreshes the materialized views and bumps the cache key.

~1 DAY • MAKES UPDATES RELIABLE

07

Alembic, Sentry, nightly pg_dump to object storage

The unglamorous three. Do them before you have users, not after you lose some.

~HALF A DAY

What not to build

Don't rewrite Flask into FastAPI, Go, or Node

You would be optimizing 2 ms out of 5,900 ms, at the cost of weeks and every working page you already have.

